

Herramientas para el desarrollo de GUI

Tema 3 – Desarrollo de Interfaces

Contenidos

1. Definiendo escenas con FXML
 - 1.1 – Cargando la escena con FXML
 - 1.2 – Uso de editores visuales: SceneBuilder
 - 1.3 – Cargando otras escenas
2. Mejorando el aspecto con CSS
3. Componentes avanzados
 - 3.1 – Tablas
 - 3.2 – Gráficas

1. Definiendo escenas con FXML

La creación, tanto de componentes como de interfaces gráficas completas conlleva mucho código repetido y muchas estructuras redundantes. Todas estas estructuras se pueden dejar al cargo de alguna herramienta que las automatice, como un Entorno de Desarrollo Integrado o IDE. Esto nos liberará de cierto trabajo aunque también tiene el inconveniente de crearnos fuertes dependencias con este IDE que pueden ser difíciles de eliminar si en algún momento queremos utilizar otra herramienta de desarrollo. Entre otros avances, la tecnología JavaFX permite la generación cómoda y sencilla de interfaces usando plantillas XML a través de esquemas propios. En este tema vamos a estudiar la creación y carga de escenas usando el lenguaje de marcado FXML.

Vamos a incorporar a nuestra plantilla básica de proyecto usada en el trimestre anterior una serie de modificaciones que nos permitirán agregar este código XML a nuestras interfaces. Uno de los cambios se da en la propia estructura de directorios del proyecto. Ahora vamos a tener un directorio nuevo, dentro de `src/main` llamado `codresources`. Dentro de este directorio tendremos una jerarquía similar a la que hay en `src/main/java` con los nombres de los paquetes. Dentro del paquete principal `com.example` encontraremos un fichero llamado `principal.fxml`.

Nuestra aplicación en blanco usada como plantilla quedará con una estructura similar a la que se muestra en el recuadro siguiente:

```

|-- pom.xml
|-- src
    |-- main
        |-- java
            |-- com
                |-- example
                    |-- App.java
                    |-- module-info.java
        |-- resources
            |-- com
                |-- example
                    |-- principal.fxml

```

La principal diferencia es que ahora no hemos creado la escena principal del **Stage** a través de instrucciones Java sino que la hemos definido en el fichero **principal.fxml** usando etiquetas XML. Usaremos el fichero FXML para crear una definición tanto del layout que queremos para nuestra escena como de los componentes que queremos en ella. Un ejemplo de lo que podría contener este fichero FXML podría ser lo que aparece en el recuadro:

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.layout.*?>
<?import javafx.scene.control.*?>
<?import javafx.geometry.Insets?>

<VBox alignment="CENTER" spacing="20.0"
    xmlns="http://javafx.com/javafx/8.0.171"
    xmlns:fx="http://javafx.com/fxml/1">

    <Label text="Etiqueta 1" />
    <Button text="Boton" />
    <Label text="Etiqueta 2" />
</VBox>

```

1.1. Cargando la escena con FXML

Ahora bien, ¿cómo cargamos este fichero en nuestra aplicación para construir los componentes JavaFX correspondientes a cada etiqueta XML del fichero? Como se puede observar en el recuadro siguiente, la clase principal de nuestra aplicación no cambia mucho. Bastaría con usar un objeto **FXMLLoader** para cargar todo el árbol de componentes. Este árbol es cargado y retornado a nosotros tras invocar al método **load()** de dicho objeto. Nos lo retorna en forma de objeto **Parent** que no es más que un objeto general de tipo nodo con el que podemos fácilmente crear nuestra escena.

Para encontrar el fichero usamos el método **getResource()**. Este método es invocado sobre **codApp.class**. Empezando por esto último, usar el atributo **class** es similar a usar el método **getClass()** sobre un objeto. Este nos retorna siempre un puntero instanciado de la propia clase del objeto. Esto suele ser útil para monitorizar aspectos de la propia clase. El método **getResource** nos retorna un objeto generado a partir de una ruta. Esta ruta siempre se construye sobre la ruta en nuestro sistema de ficheros, a partir del directorio donde se guarda el fichero **.class** de esta clase. Si analizamos los directorios bajo **target** en nuestro proyecto veremos que en el directorio **target/classes/com/example** encontramos tanto **App.class** como **principal.fxml**.

```

public class App extends Application {

    private static Scene scene;

    @Override
    public void start(Stage stage) throws IOException {

        FXMLLoader loader = new FXMLLoader(App.class.getResource("principal.fxml"));
        Parent root = loader.load();

        scene = new Scene(root, 640, 480);
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}

```

1.2. Uso de editores visuales: SceneBuilder

La aplicación más conocida para el desarrollo de interfaces gráficas con JavaFX que es SceneBuilder (ver figura 1). Esta aplicación permite el diseño sencillo de una interfaz gráfica basada en JavaFX a través del arrastrado visual de componentes. Por debajo, SceneBuilder nos irá construyendo un fichero FXML al que luego podemos incorporarle otros elementos, bien desde la propia configuración gráfica a través del panel derecho o bien directamente editando estos ficheros con nuestro entorno de programación.

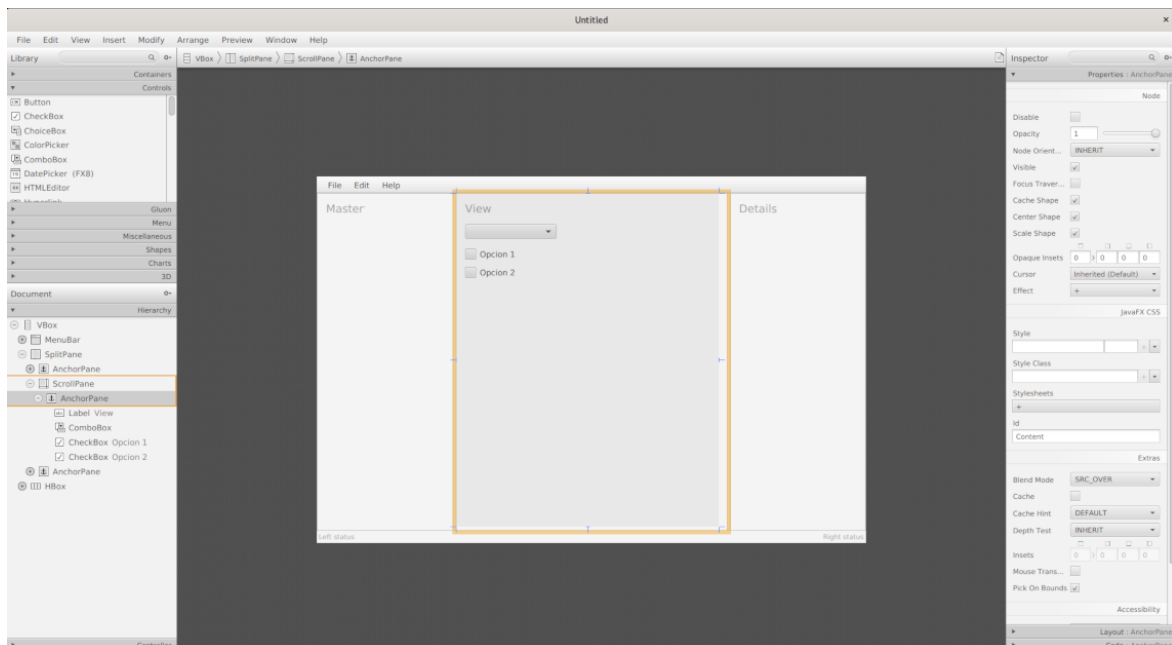


Figura 1: Aspecto de la interfaz principal de SceneBuilder

Para crear nuestra primera interfaz podemos abrir en SceneBuilder el fichero `principal.fxml` de nuestro proyecto plantilla (`di-0??`). Allí lo primero que vemos es un simple panel con una sola etiqueta. En el panel de la izquierda podemos elegir nodos divididos en: contenedores, controles, menús, ... Pulsaremos sobre uno de ellos, por ejemplo un botón y lo arrastraremos a la zona central. Una vez colocado, en el panel derecho podemos estudiar todas sus propiedades.

Una de las primeras propiedades interesantes que podemos asignarle y a la que más adelante nos referiremos es un identificador. Desplazamos hacia abajo el panel derecho hasta encontrar el cuadro de texto que nos permita asignarle un nombre a esta propiedad y lo escribiremos. Si salvamos ahora los cambios y abrimos el fichero FXML con un editor de texto veremos que, entre otros cambios veremos también la nueva etiqueta para el botón con su identificador asignado:

```
<Button id="miboton" mnemonicParsing="false" text="Button" />
```

En todo momento somos libres de poder editar el fichero FXML directamente desde nuestro editor de programación. No es necesario el uso de SceneBuilder para esta tarea aunque, si la interfaz empieza a complicarse es muy recomendable su utilización.

1.2.1. Controladores y eventos

Enlazar eventos a través de FXML es totalmente posible pero hemos de aplicar un patrón ligeramente diferente. El código del evento puede ser enlazado al componente programado en Java o, incluso, en otros lenguajes de *scripting* como Javascript, Python o Groovy pero nosotros solo usaremos el primero. Siguiendo lo aprendido en MVC lo primero que necesitamos para implementar el código de nuestros manejadores es un o varias clases controladoras. Los controladores son objetos Java que llevan la funcionalidad de la interfaz definida en el fichero FXML. Hemos de fijarnos que se llaman así debido a la terminología estudiada en temas anteriores respecto del patrón MVC. Un controlador básico puede tener la siguiente estructura:

```
package com.example;

import javafx.fxml.FXML;
import java.io.IOException;

public class Controlador {

    @FXML
    private void manejarEvento() throws IOException {
        System.out.println("Este es mi primer manejador en FXML");
    }
}
```

Dentro del método `manejadorEvento()` escribiremos el código del manejador de forma usual. Es importante no olvidar la anotación sobre el método para que todo funcione. Ahora veremos como enganchamos el manejador con el componente definido en el fichero FXML. Esta asociación entre el FXML y la clase del controlador se realiza especificando el nombre de la clase como el valor del atributo `fx:controller` en el elemento raíz del FXML: `fx:controller='com.example.Controlador'`.

Una vez hecho esto, para terminar asociar el manejador a un nodo de nuestra interfaz usaremos, en este caso, el atributo `onAction` seguido del nombre del método que gestiona ese evento precedido por el carácter `#`.

```
<Button fx:id="boton" text="Pulsa" onAction="#manejadorEvento"/>
```

Si queremos utilizar algún elemento de la interfaz dentro de un controlador lo ideal es que le marquemos con un identificador usando el atributo `fx:id`. Por ejemplo podemos querer tener una etiqueta cuyo contenido queramos luego recuperar en un controlador para que sea procesado. Ahora nos fijamos en el código del controlador:

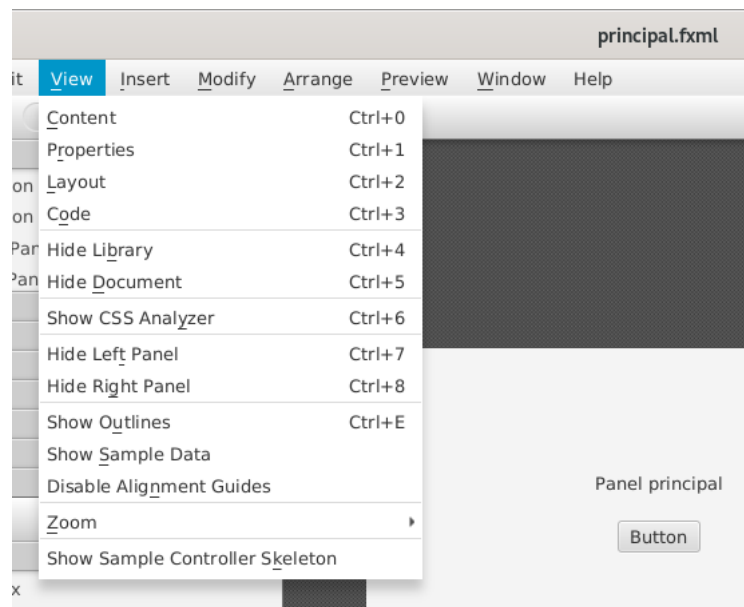
```
public class Controlador{

    @FXML
    private Label texto;

    @FXML
    private void manejadorEvento() throws IOException {
        System.out.println("texto.getText());
    }
}
```

Hemos declarado un atributo privado de tipo `Label` que se llama igual que la ID que le hemos puesto a dicha etiqueta en el FXML. Esto hará que automáticamente el objeto creado en el FXML se vincule a este atributo y pueda ser usado sin problemas en el método `manejadorEvento()`.

Para terminar indicaremos que en SceneBuilder podemos conseguir rápidamente un esqueleto de un controlador asociado a una vista. Tras terminar por completo su distribución, pulsaremos sobre el menú **View** y elegiremos la última opción con el texto: **Show Sample Controller Skeleton**.



Cuando el controlador es cargado por primera vez, el motor de JavaFX ejecuta el método `initialize()` si es que este ha sido implementado en su interior. Puedes implementar y utilizar este método para incluir en el código que quieres que se ejecute siempre al cargarse el controlador asociado a esa escena. Este método se implementa en cualquier parte del controlador y no necesita incorporar parámetros:

```
public void initialize() { ... }
```

1.3. Cargando otras escenas

Cargar otras escenas en el stage es tan sencillo como cargar un segundo árbol de nodos y enlazarlo a la raíz de nuestra escena actual. Esto modificará en tiempo de ejecución toda la apariencia de la interfaz. Podemos realizar esta simple carga igual que hicimos la del árbol inicial en el propio manejador del evento en el controlador FXML★ :

```
public class Controlador{

    @FXML
    private void manejadorEvento() throws IOException {

        FXMLLoader loader = new FXMLLoader(App.class.getResource("secundaria.fxml"));
        Parent root = loader.load();

        App.getScene().setRoot(root);
    }
}
```

También podríamos tener otra estructura como una colección de árboles de nodos precargados previamente e ir cargando uno u otro según nos indiquen.

2. Mejorando el aspecto con CSS

Uno de los principales avances de JavaFX es permitir una rápida personalización del aspecto de nuestra interfaz gráfica usando hojas de estilo en cascada o CSS. La idea fundamental de este mecanismo está importada del ámbito web y de HTML. Asociaremos identificadores a nuestros componentes gráficos y les vincularemos reglas CSS que indiquen como queremos que estos sean visualizados.

La mejor manera de realizar esta vinculación es crear una hoja de estilo separada y almacenada en el directorio `src/main/resources` de nuestro proyecto. De esta forma Maven tendrá siempre localizada la hoja de estilo no tendremos problemas de localización. Tras compilar nuestro proyecto, esta hoja será situada directamente en el directorio `target/classes`, fuera de la estructura de paquetes. Para usar las propiedades de esta hoja la cargaremos en nuestra escena usando el método `getStylesheets().add()`*. Partimos de una simple hoja de estilo donde definimos una propiedad para los elementos de la clase `.button`:

```
.button{
    -fx-background-color:#90EE90;
}
```

Ahora en nuestra clase Java hemos de cargar esta hoja de estilo a nuestra escena y vincular a los componentes que queramos que luzcan este aspecto, esta regla CSS.

```
VBox raiz = new VBox();
Scene primera = new Scene(raiz,300,300);
ventana.setScene(primera);

primera.getStylesheets().add(getClass()
    .getResource("/estilos.css")
    .toExternalForm());

Button boton = new Button("Pulsa");
raiz.getChildren().add(boton);

Button boton2 = new Button("Guarda");
raiz.getChildren().add(boton2);
```

En JavaFX todos los componentes pertenecen a una clase CSS del mismo nombre que su clase Java. En el ejemplo anterior, los botones pertenecerán a la clase `Button`). En caso que querer que un

componente tenga una clase CSS concreta podemos añadirsele a su array de clases que obtendremos con el método `getStyleClass()`. Añadimos a uno de los botones del ejemplo anterior la clase CSS «pulsado»:

```
boton2.getStyleClass().add("pulsado");
```

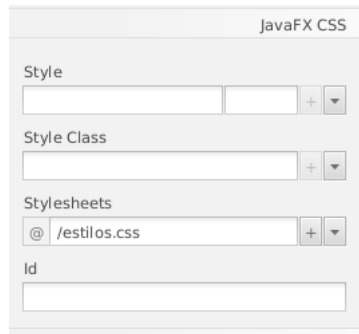


Figura 2: Como vincular una CSS a un elemento

Igualmente podemos asignar visualizaciones concretas a objetos a través de identificadores CSS igual que en el ámbito web. Para usar un selector CSS de tipo identificador usaremos el nombre ese precedido de un caracter `#` en nuestra CSS. En el código, para asignarle un identificador a un nodo usaremos el método `setId()` sobre dicho nodo. Si ambos identificadores coinciden, a este nodo se le aplicarán estas reglas de CSS. Es importante remarcar también que JavaFX no tiene incorporado un procesador completo de CSS por lo que ciertos elementos de esta tecnología pueden que no estén incorporados. Para conocer las reglas con las que podemos cambiar el aspecto de cada componente, lo ideal es leer la documentación oficial en Internet buscando el documento titulado «*JavaFX CSS Reference Guide*»¹.

Esta vinculación también podemos hacerla de forma sencilla desde SceneBuilder. Teniendo en cuenta que tenemos situado nuestro fichero de estilos en el directorio **resources** como hemos explicado anteriormente, seleccionamos en el editor el componente raíz de nuestra escena. En el panel derecho buscamos un campo de texto con el texto **Stylesheets** y allí introducimos o exploramos hasta encontrar la ruta de nuestra hoja de estilos. Es importante que la ruta la comencemos con el caracter `'/'` tal y como se ve en la figura 2.

Tras hacer esto y guardar los cambios, reconstruimos nuestra aplicación y estos deben hacerse efectivos. Si queremos que además, SceneBuilder cargue nuestra hoja de estilos en la previsualización para trabajar con una versión «más real» de nuestra vista, iremos al menú **Preview** y el submenú **Scene Style Sheets** la añadiremos.

3. Componentes avanzados

Vamos a estudiar ahora algún componente más sobre los ya detallados en temas anteriores. Hablaremos sobre las tablas, elementos para mostrar colecciones de datos algo más complejas que las listas y sobre los componentes de tipo lienzo o *canvas* que nos permitirán renderizar dibujos de trazo libre y figuras geométricas.

¹Podéis encontrar la última versión de este documento enlazada en la web de la asignatura

3.1. Tablas

Para construir una tabla hemos de tener en cuenta los mecanismos ya estudiados como son la colecciones observables y las propiedades. Es importante remarcar la importancia de que, si queremos que los objetos de una clase de nuestro modelo aparezcan en una tabla, esta ha debido ser construida usando un sistema de propiedades similar al explicado con anterioridad. Vamos a recordar la clase `Alumno` que creamos en esa pasada sección:²

```
public class Alumno {

    private final SimpleStringProperty nombre;
    private final IntegerProperty edad;

    public Alumno(String nombre,int edad){
        this.nombre=new SimpleStringProperty(nombre);
        this.edad=new SimpleIntegerProperty(edad);
    }

    ...
}
```

Las tablas se basan en la clase `TableView`[★]. Esta clase estará formada por columnas de la clase `TableColumn`. En estas clases columnas es donde mapearemos que propiedad de la clase modelo vamos a visualizar. Este mapeo entre columna y propiedad del modelo lo haremos a través del método `setCellValueFactory()` tal y como se ve en el ejemplo inferior. Este método recibirá un objeto de tipo `PropertyValueFactory()` con el nombre de la propiedad del objeto del modelo que queramos mapear en esa columna.

```
TableView<Libro> table = new TableView<Libro>();

TableColumn<Alumno, String> nombre = new TableColumn<>("Nombre");
nombre.setCellValueFactory(new PropertyValueFactory<>("nombre"));

TableColumn<Alumno, Integer> edad = new TableColumn<>("Edad");
edad.setCellValueFactory(new PropertyValueFactory<>("edad"));

...

// aula es un ArrayList<Alumno> donde están guardados todos mis objetos
// del modelo

ObservableList<Alumno> datos = FXCollections.observableArrayList(aula);
table.setItems(datos);
table.getColumns().addAll(nombre,edad);
```

Para terminar, suponemos que nuestros datos del modelo están guardados en un `ArrayList` llamado `aula`. Este lo convertimos a `ObservableList`, que es el tipo de datos que manejará la tabla y se lo insertamos con el método `setItems()`. Por último agregamos los objetos `TableColumn` a la tabla.

En el caso que tras se cargada y visualizada queramos incluir nuevos datos a la tabla basta con recuperar el objeto `TableView` llamado a su método `getItems()` y sobre el tipo coleccionable que este nos retorna llamara a `add()`. Por ejemplo:

```
table.getItems().add(new Alumno("Nuevo alumno",00));
```

²En el caso de que no queramos modificar nuestras clases del modelo siempre podremos crear un *wrapper* o envoltorio que haga de puente.

3.2. Gráficas

Para visualizar gran cantidad de datos mejor que una tabla puede resultar una gráfica. Con este elemento, perderemos información de grano fino sobre la colección de datos pero ganaremos visión de conjunto sobre un gran conjunto de datos. En JavaFX tenemos varios controles preconstruidos para visualizar gráficas y este año estudiaremos dos: las gráficas lineales y las circulares.

Los gráficos lineales están formados por dos ejes: el vertical o eje Y y el horizontal o eje X. Los valores introducidos deben tener ser situados proyectados sobre ambos dándoles un valor entre los rangos de ambos ejes. Finalmente se dibujará una línea que una todos los puntos quedándonos algo como se muestra en la figura.



Veamos como construir un ejemplo similar★. Todos los objetos usados en esta sección forman parte del subpaquete `javafx.scene.chart`. Para la creación de los ejes usaremos el objeto `NumberAxis` y con dos objetos de este tipo creamos la gráfica de tipo `LineChart`.

```
final NumberAxis ejeX = new NumberAxis();
final NumberAxis ejeY = new NumberAxis();

final LineChart<Number,Number> lineChart = new LineChart<Number,Number>(ejeX,ejeY);
```

Una vez hecho esto solo nos queda poblar la gráfica de puntos de valor para que luego estos sean unidos. Lo primero será crear una serie numérica e introducir en ella estos puntos. Para introducir un nuevo valor usaremos el método estático `XYChart.Data()` que recibe dos números. El objeto retornado lo introduciremos en las series tal y como se ve en el código siguiente:

```
XYChart.Series series = new XYChart.Series();

series.getData().add(new XYChart.Data(1, 23));
series.getData().add(new XYChart.Data(2, 14));
series.getData().add(new XYChart.Data(3, 15));
// ... mas datos
```

Solo nos quedaría asignar al gráfico creado anteriormente la serie de datos construida y una vez hecho esto enlazar el nodo de tipo gráfico a nuestro árbol de nodos donde queramos.

```
lineChart.getData().add(series);
```

Una gráfica circular es similar pero despliega sus datos en forma radial★, aportando porciones a cada dato. De ahí que se llame también gráfica de pastel o *pie chart*. La estructura de esta gráfica es



mucho más sencilla y solo requiere de un observableList que hemos de construir con los datos de la gráfica. Hemos de tener en cuenta que ahora los datos representan porcentajes de áreas radiales. Con estos contruidos crearemos un objeto de tipo `PieChart` al que asignaremos este conjunto de datos:

```
ObservableList<PieChart.Data> pieChartData =
FXCollections.observableArrayList(
    new PieChart.Data("Linux", 30),
    new PieChart.Data("MacOs", 5),
    new PieChart.Data("Windows", 50),
    new PieChart.Data("Otros Unix", 5),
    new PieChart.Data("Android", 10));

PieChart chart = new PieChart(pieChartData);
```

Lista de ejemplos

1	Ejemplo de recarga de una escena para cambiar su apariencia (di-054)	5
2	Ejemplo de uso de integración de CSS a través del código fuente (di-052)	6
3	Ejemplo de uso de TableView (di-042)	8
4	Ejemplo de gráfica lineal (di-055)	9
5	Ejemplo de gráfica circular (di-056)	9

Actividades

1. Usa SceneBuilder para crear un formulario de entrada de datos personales sencillo personalizando márgenes y alineamientos a través de la interfaz del editor.
2. Utiliza SceneBuilder para crear una vista en la que calculemos el precio total de unos items o artículos que muestres de la forma que consideres. Debemos poder seleccionar de alguna manera un subconjunto de estos artículos y la vista nos tiene que ofrecer el precio total de los artículos seleccionados. Los artículos deben ser cargados desde el modelo y pueden variar así que no podrás diseñar tu interfaz considerando un conjunto fijo de artículos (pista: utiliza el método `initialize()` del controlador).
3. Integra reglas CSS sencillas con algunas propiedades básicas en alguno de los proyectos anteriores y modifica su aspecto a través de la asignación de identificadores. Puedes probar a hacerlo de forma individual o integrando una hoja de estilos completa.
4. Construye una aplicación que visualice tanto un formulario para introducir notas de alumnos (los campos que debes introducir son nombre, apellido y nota) y una tabla donde se muestren estos datos de todos los alumnos introducidos. La nota será un cadena entre los valores: suspenso, aprobado, notable y sobresaliente.